Name : Manoj Kumar.M
Class : IT Batch 2
Roll no : 2025115304

DS Homework Set – 4

1) Implement a Stack using Queue data structure.

```
#include <stdio.h>
#define MAX 100
int queue[MAX];
int front = -1, rear = -1;
void enqueue(int x)
{
  if (rear == MAX - 1)
    printf("Queue Overflow\n");
  else
  {
    if (front == -1)
      front = 0;
    queue[++rear] = x;
  }
}
int dequeue()
{
  if (front == -1 || front > rear)
    return -1;
  return queue[front++];
}
void push(int x)
{
  int i, n;
  n = rear - front + 1;
  enqueue(x);
  for (i = 0; i < n; i++)
    enqueue(dequeue());
}
int pop()
{
  if (front == -1 || front > rear)
    return -1;
  return dequeue();
}
void display()
{
  int i;
  if (front == -1 || front > rear)
  {
    printf("Stack is empty\n");
    return;
  }
  printf("Stack elements: ");
```

```
  for (i = rear; i >= front; i--)
    printf("%d ", queue[i]);
  printf("\n");
}
int main()
{
  int choice, x;
  while (1)
  {
    printf("\n1. Push\n2. Pop\n3. Display\n4. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
      case 1:
        printf("Enter element: ");
        scanf("%d", &x);
        push(x);
        break;
      case 2:
        x = pop();
        if (x == -1)
          printf("Stack Underflow\n");
        else
          printf("Popped element: %d\n", x);
        break;
      case 3:
        display();
        break;
      case 4:
        return 0;
      default:
        printf("Invalid choice\n");
    }
  }
}
```

Output :

```
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 10

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 20
```

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 30

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack elements: 30 20 10

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped element: 30

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack elements: 20 10

3) Implement a Queue using Stack data structure.

```
#include <stdio.h>
#define MAX 100
int stack1[MAX], stack2[MAX];
int top1 = -1, top2 = -1;
void push1(int x)
{
  if (top1 == MAX - 1)
    printf("Queue Overflow\n");
  else
    stack1[++top1] = x;
}
int pop1()
{
  return stack1[top1--];
}
void push2(int x)
{
  stack2[++top2] = x;
}
int pop2()
{
  return stack2[top2--];
}
```

```
void enqueue(int x)
{
  push1(x);
}
int dequeue()
{
  int x;
  if (top1 == -1 && top2 == -1)
    return -1;
  if (top2 == -1)
  {
    while (top1 != -1)
    {
      x = pop1();
      push2(x);
    }
  }
  return pop2();
}
void display()
{
  int i;
  if (top1 == -1 && top2 == -1)
  {
    printf("Queue is empty\n");
    return;
  }
  printf("Queue elements: ");
  for (i = top2; i >= 0; i--)
    printf("%d ", stack2[i]);
  for (i = 0; i <= top1; i++)
    printf("%d ", stack1[i]);
  printf("\n");
}
int main()
{
  int choice, x;
  while (1)
  {
    printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
      case 1:
        printf("Enter element: ");
        scanf("%d", &x);
        enqueue(x);
        break;
      case 2:
        x = dequeue();

        if (x == -1)
```

```
            printf("Queue Underflow\n");
          else
            printf("Deleted element: %d\n", x);
          break;
        case 3:
          display();
          break;
        case 4:
          return 0;
        default:
          printf("Invalid choice\n");
      }
    }
}
```

Output :

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 10

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 20

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 30

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Queue elements: 10 20 30

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Deleted element: 10

1. Enqueue
```

2. Dequeue
3. Display
4. Exit
Enter choice: 3
Queue elements: 20 30

3) Write a C program using stack data structure to manage the history of web pages visited.

```
#include <stdio.h>
#include <string.h>
#define MAX 10
char stack[MAX][100];
int top = -1;
void visit(char page[])
{
  if (top == MAX - 1)
    printf("History is full\n");
  else
  {
    strcpy(stack[++top], page);
    printf("Page visited: %s\n", page);
  }
}
void back()
{
  if (top == -1)
    printf("No history available\n");
  else
  {
    printf("Going back from: %s\n", stack[top]);
    top--;
    if (top != -1)
      printf("Current page: %s\n", stack[top]);
    else
      printf("No page in history\n");
  }
}
void display()
{
  int i;
  if (top == -1)
  {
    printf("History is empty\n");
    return;
  }
  printf("Browser History:\n");
  for (i = top; i >= 0; i--)
    printf("%s\n", stack[i]);
}
int main()
{
  int choice;
  char page[100];
```

```
  while (1)
  {
    printf("\n1. Visit Page\n");
    printf("2. Back\n");
    printf("3. Display History\n");
    printf("4. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
      case 1:
        printf("Enter web page: ");
        scanf("%s", page);
        visit(page);
        break;
      case 2:
        back();
        break;
      case 3:
        display();
        break;
      case 4:
        return 0;
      default:
        printf("Invalid choice\n");
    }
  }
}
```

Output :

```
1. Visit Page
2. Back
3. Display History
4. Exit
Enter choice: 1
Enter web page: google.com
Page visited: google.com

1. Visit Page
2. Back
3. Display History
4. Exit
Enter choice: 1
Enter web page: youtube.com
Page visited: youtube.com

1. Visit Page
2. Back
3. Display History
4. Exit
Enter choice: 1
Enter web page: github.com
```

Page visited: github.com

1. Visit Page
2. Back
3. Display History
4. Exit
Enter choice: 3
Browser History:
github.com
youtube.com
google.com

1. Visit Page
2. Back
3. Display History
4. Exit
Enter choice: 2
Going back from: github.com
Current page: youtube.com

4) Write a C program to sort a stack using temporary stack. (Implement using stack)

```
#include <stdio.h>
#define MAX 100
int stack[MAX], temp[MAX];
int top = -1, ttop = -1;
void push(int s[], int *top, int x)
{
  if (*top == MAX - 1)
    printf("Stack Overflow\n");
  else
    s[++(*top)] = x;
}
int pop(int s[], int *top)
{
  if (*top == -1)
    return -1;

  return s[(*top)--];
}
void sortStack()
{
  int x, y;
  while (top != -1)
  {
    x = pop(stack, &top);
    while (ttop != -1 && temp[ttop] > x)
    {
      y = pop(temp, &ttop);
      push(stack, &top, y);
    }
    push(temp, &ttop, x);
  }
```

```
  while (ttop != -1)
  {
    x = pop(temp, &ttop);
    push(stack, &top, x);
  }
}
void display()
{
  int i;
  if (top == -1)
  {
    printf("Stack is empty\n");
    return;
  }
  printf("Stack elements: ");
  for (i = top; i >= 0; i--)
    printf("%d ", stack[i]);
  printf("\n");
}
int main()
{
  int choice, x;
  while (1)
  {
    printf("\n1. Push\n");
    printf("2. Sort\n");
    printf("3. Display\n");
    printf("4. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
      case 1:
        printf("Enter element: ");
        scanf("%d", &x);
        push(stack, &top, x);
        break;
      case 2:
        sortStack();
        printf("Stack sorted\n");
        break;
      case 3:
        display();
        break;
      case 4:
        return 0;
      default:
        printf("Invalid choice\n");
    }
  }
}
```

Output :

1. Push
2. Sort
3. Display
4. Exit
Enter choice: 1
Enter element: 30

1. Push
2. Sort
3. Display
4. Exit
Enter choice: 1
Enter element: 10

1. Push
2. Sort
3. Display
4. Exit
Enter choice: 1
Enter element: 40

1. Push
2. Sort
3. Display
4. Exit
Enter choice: 1
Enter element: 20

1. Push
2. Sort
3. Display
4. Exit
Enter choice: 2
Stack sorted

1. Push
2. Sort
3. Display
4. Exit
Enter choice: 3
Stack elements: 10 20 30 40

5) The stock span problem is a financial problem where we have a series of daily price for a stock, denoted by an array and the task is to calculate the span of the stock's price for all days. The span of a stock's price for i-th day gives the maximum number of consecutive days leading up to i-th day (including current day) where stock price is less than or equal to its price on day i.

```
#include <stdio.h>
#define MAX 100
int stack[MAX];
int top = -1;
void push(int x)
```

```
{
  stack[++top] = x;
}
int pop()
{
  return stack[top--];
}
int main()
{
  int n, i, price[MAX], span[MAX];
  printf("Enter number of days: ");
  scanf("%d", &n);
  printf("Enter stock prices: ");
  for (i = 0; i < n; i++)
    scanf("%d", &price[i]);
  for (i = 0; i < n; i++)
  {
    while (top != -1 && price[stack[top]] <= price[i])
      pop();
    if (top == -1)
      span[i] = i + 1;
    else
      span[i] = i - stack[top];
    push(i);
  }
  printf("Stock span: ");
  for (i = 0; i < n; i++)
    printf("%d ", span[i]);
  return 0;
}
```

Output :

```
Enter number of days: 7
Enter stock prices: 100 80 60 70 60 75 85
Stock span: 1 1 1 2 1 4 6
```

6) Given an array, for each element find the nearest element to the right which has a frequency greater than the current element. If no such element exists, return -1. Implement using stack.

```
#include <stdio.h>
#define MAX 100
int stack[MAX];
int top = -1;
void push(int x)
{
  stack[++top] = x;
}
int pop()
{
  return stack[top--];
}
int main()
```

```
{
  int n, i, j;
  int a[MAX], freq[MAX], result[MAX];
  printf("Enter number of elements: ");
  scanf("%d", &n);
  printf("Enter elements: ");
  for (i = 0; i < n; i++)
    scanf("%d", &a[i]);
  for (i = 0; i < n; i++)
  {
    freq[i] = 0;
    for (j = 0; j < n; j++)
    {
      if (a[i] == a[j])
        freq[i]++;
    }
  }
  for (i = n - 1; i >= 0; i--)
  {
    while (top != -1 && freq[stack[top]] <= freq[i])
      pop();
    if (top == -1)
      result[i] = -1;
    else
      result[i] = a[stack[top]];
    push(i);
  }
  printf("Output: ");
  for (i = 0; i < n; i++)
    printf("%d ", result[i]);
  return 0;
}
```

Output :

Enter number of elements: 6
Enter elements: 2 1 1 3 2 1
Output: 1 -1 -1 2 1 -1

7) Given a pattern containing only I's and D's. I for increasing and D for decreasing. Devise an algorithm using stack to print the lexicographically smallest number following that pattern. Digits from 1–9 and digits can't repeat.

```
#include <stdio.h>
#include <string.h>
#define MAX 100
int stack[MAX];
int top = -1;
void push(int x)
{
  stack[++top] = x;
}
int pop()
```

```
{
  return stack[top--];
}
int main()
{
  char pattern[MAX];
  int i, n, num;
  printf("Enter pattern: ");
  scanf("%s", pattern);
  n = strlen(pattern);
  printf("Smallest number: ");
  for (i = 0; i <= n; i++)
  {
    push(i + 1);
    if (i == n || pattern[i] == 'I')
    {
      while (top != -1)
      {
        num = pop();
        printf("%d", num);
      }
    }
  }
  return 0;
}
```

Output :

```
Enter pattern: DIDDI
Smallest number: 21435
```

8) Given an array of digits (values are from 0 to 9), find the minimum possible sum of two numbers formed from digits of the array. All digits of the given array must be used to form the two numbers. Use Queue for implementation.

```
#include <stdio.h>
#define MAX 100
int queue[MAX];
int front = 0, rear = -1;
void enqueue(int x)
{
  queue[++rear] = x;
}
int dequeue()
{
  return queue[front++];
}
int main()
{
  int n, i, j, temp;
  int num1 = 0, num2 = 0;
  printf("Enter number of digits: ");
  scanf("%d", &n);
```

```
  printf("Enter digits: ");
  for (i = 0; i < n; i++)
  {
    scanf("%d", &queue[i]);
    rear++;
  }
  for (i = 0; i < n - 1; i++)
  {
    for (j = i + 1; j < n; j++)
    {
      if (queue[i] > queue[j])
      {
        temp = queue[i];
        queue[i] = queue[j];
        queue[j] = temp;
      }
    }
  }
  if (queue[0] == 0)
  {
    for (i = 0; i < n; i++)
    {
      if (queue[i] != 0)
      {
        temp = queue[0];
        queue[0] = queue[i];
        queue[i] = temp;
        break;
      }
    }
  }
  for (i = 0; i < n; i++)
  {
    if (i % 2 == 0)
      num1 = num1 * 10 + queue[i];
    else
      num2 = num2 * 10 + queue[i];
  }
  printf("Minimum sum: %d", num1 + num2);
  return 0;
}
```

Output :

```
Enter number of digits: 6
Enter digits: 6 8 4 5 2 3
Minimum sum: 604
```

9) Given an array and a positive integer K, find the first negative integer for each window (contiguous subarray) of size K. If a window does not contain a negative integer, print 0. Implement using Queue.

```
#include <stdio.h>
#define MAX 100
```

```
int queue[MAX];
int front = 0, rear = -1;
void enqueue(int x)
{
  queue[++rear] = x;
}
int dequeue()
{
  return queue[front++];
}
int main()
{
  int a[MAX], n, k;
  int i, j, found;
  printf("Enter number of elements: ");
  scanf("%d", &n);
  printf("Enter elements: ");
  for (i = 0; i < n; i++)
    scanf("%d", &a[i]);
  printf("Enter window size: ");
  scanf("%d", &k);
  printf("Output: ");
  for (i = 0; i <= n - k; i++)
  {
    front = 0;
    rear = -1;
    found = 0;
    for (j = i; j < i + k; j++)
    {
      if (a[j] < 0)
      {
        enqueue(a[j]);
        found = 1;
        break;
      }
    }
    if (found)
      printf("%d ", dequeue());
    else
      printf("0 ");
  }
  return 0;
}
```

Output :

Enter number of elements: 5
Enter elements: -8 2 3 -6 1
Enter window size: 2
Output: -8 0 -6 -6

10) Given a matrix of dimension n × n where each cell in the matrix can have values 0, 1, or 2 which has the following meaning:

- **0: Empty cell**
- **1: Cell having fresh oranges**
- **2: Cell having rotten oranges**

The task is to find the minimum time required so that all oranges become rotten. A rotten orange at index (i,j) can rot its neighbours (up, down, left and right). If impossible to rot every orange, return -1. Use Queue for implementation.

```
#include <stdio.h>
#define MAX 100
struct Node
{
  int r, c, time;
};
struct Node queue[MAX * MAX];
int front = 0, rear = -1;
void enqueue(int r, int c, int time)
{
  queue[++rear].r = r;
  queue[rear].c = c;
  queue[rear].time = time;
}
struct Node dequeue()
{
  return queue[front++];
}
int main()
{
  int a[MAX][MAX];
  int n, i, j;
  int fresh = 0, maxTime = 0;
  int dr[] = {-1, 1, 0, 0};
  int dc[] = {0, 0, -1, 1};
  printf("Enter matrix size: ");
  scanf("%d", &n);
  printf("Enter matrix:\n");
  for (i = 0; i < n; i++)
  {
    for (j = 0; j < n; j++)
    {
      scanf("%d", &a[i][j]);
      if (a[i][j] == 1)
        fresh++;
      if (a[i][j] == 2)
        enqueue(i, j, 0);
    }
  }
  while (front <= rear)
  {
    struct Node cur = dequeue();
    for (i = 0; i < 4; i++)
    {
      int nr = cur.r + dr[i];
      int nc = cur.c + dc[i];
```

```
        if (nr >= 0 && nr < n &&
            nc >= 0 && nc < n &&
            a[nr][nc] == 1)
        {
            a[nr][nc] = 2;
            fresh--;
            enqueue(nr, nc, cur.time + 1);
            if (cur.time + 1 > maxTime)
                maxTime = cur.time + 1;
        }
      }
    }
    if (fresh > 0)
      printf("Minimum time: -1\n");
    else
      printf("Minimum time: %d\n", maxTime);
    return 0;
}
```

Output :

```
Enter matrix size: 3
Enter matrix:
2 1 0
1 0 1
1 0 0
Minimum time: 2
```